RV College of Engineering®

Big Data Technologies – Unit 4

Dr. Narasimha Swamy S | Department of AIML

Go, Change the World

Outline

- Flume
  - Introduction
  - Installing Flume
- Transactions and Reliability
  - Batching
- The HDFS Sink
  - Partitioning and Interceptors File Formats
- Fan Out
  - Delivery Guarantees
  - Replicating and Multiplexing Selectors
- Distribution: Agent Tiers
  - Delivery Guarantees
- Sink Groups
  - Integrating Flume with Applications
  - Component Catalog

Flume

Introduction

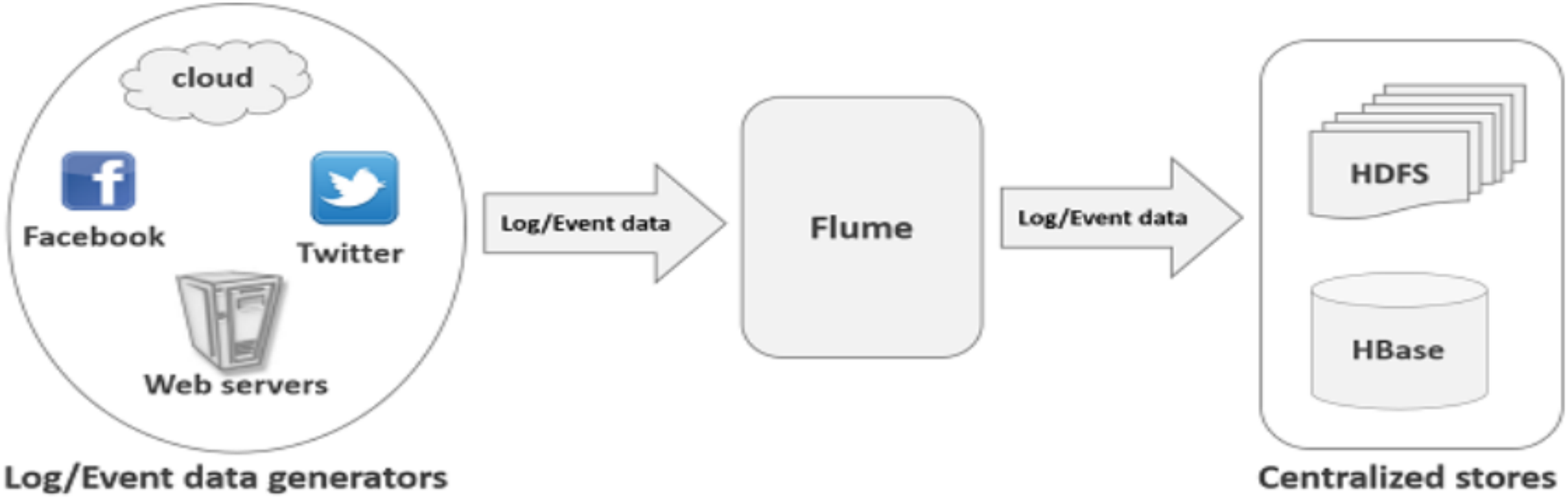
- Apache Flume is a distributed system for collecting, aggregating, and transporting large volumes of data
- Primarily used for log data ingestion into Hadoop (HDFS)
- Why Flume?
  - Handles high-volume
  - Streaming data
  - Moves data reliably and efficiently
  - Supports real-time data ingestion
- Core Architecture
  - Source → collects data
  - Channel → temporary storage (Buffer)
  - Sink → sends data to destination (HDFS, HBase)
- Data Flow

  Source → Channel → Sink → HDFS

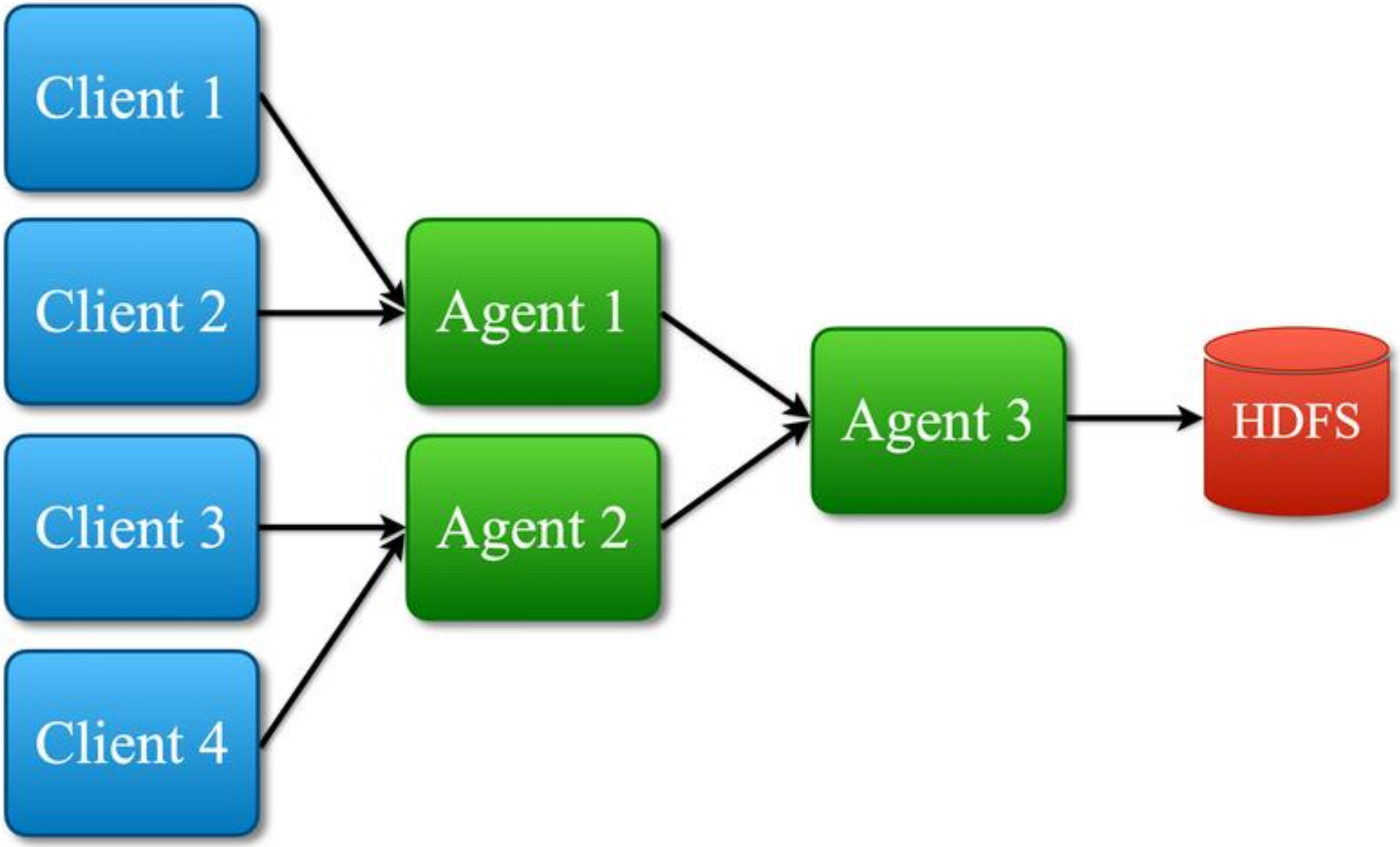
Applications

- Web server logs
- Application logs
- Event streaming

Introduction (Contd.)



Architecture of Flume



Installing Flume

- Hadoop is built for processing very large datasets.
- Often it is assumed that the data is already in HDFS or can be copied there in bulk.
- However, there are many systems that don't meet this assumption.
- They produce streams of data that we would like to aggregate, store, and analyze using Hadoop and these are the systems that Apache Flume is an ideal fit for.
- Flume is designed for high-volume ingestion into Hadoop of event-based data.
- Flume is a standard, simple, robust, flexible, and extensible tool for data ingestion from various data producers (webservers) into Hadoop.
- The canonical example is using Flume to collect logfiles from a bank of web servers, then moving the log events from those files into new aggregated files in HDFS for processing.
- The usual destination (or sink in Flume parlance) is HDFS. However, Flume is flexible enough to write to other systems, like HBase or Solr
- Apache Flume is a tool/service/data ingestion mechanism for collecting aggregating and transporting large amounts of streaming data such as log files, events (etc.) from various sources to a centralized data store.

Installing Flume (Contd.)

- Hadoop is built for processing very large datasets.
- Often it is assumed that the data is already in HDFS or can be copied there in bulk.
- However, there are many systems that don’t meet this assumption.
- They produce streams of data that we would like to aggregate, store, and analyze using Hadoop and these are the systems that Apache Flume is an ideal fit for.
- Flume is designed for high-volume ingestion into Hadoop of event-based data.
- Flume is a standard, simple, robust, flexible, and extensible tool for data ingestion from various data producers (webservers) into Hadoop.
- The canonical example is using Flume to collect logfiles from a bank of web servers, then moving the log events from those files into new aggregated files in HDFS for processing.
- The usual destination (or sink in Flume parlance) is HDFS. However, Flume is flexible enough to write to other systems, like HBase or Solr
- Apache Flume is a tool/service/data ingestion mechanism for collecting aggregating and transporting large amounts of streaming data such as log files, events (etc.) from various sources to a centralized data store.

Installing Flume (Contd.)

- Hadoop is built for processing very large datasets.
- Often it is assumed that the data is already in HDFS or can be copied there in bulk.
- However, there are many systems that don’t meet this assumption.
- They produce streams of data that we would like to aggregate, store, and analyze using Hadoop and these are the systems that Apache Flume is an ideal fit for.
- Flume is designed for high-volume ingestion into Hadoop of event-based data.
- Flume is a standard, simple, robust, flexible, and extensible tool for data ingestion from various data producers (webservers) into Hadoop.
- The canonical example is using Flume to collect logfiles from a bank of web servers, then moving the log events from those files into new aggregated files in HDFS for processing.
- The usual destination (or sink in Flume parlance) is HDFS. However, Flume is flexible enough to write to other systems, like HBase or Solr
- Apache Flume is a tool/service/data ingestion mechanism for collecting aggregating and transporting large amounts of streaming data such as log files, events (etc.) from various sources to a centralized data store.

Architecture of Flume (Contd.)

- To use Flume, we need to run a Flume agent, which is a long-lived Java process that runs sources and sinks, connected by channels.
- A source in Flume produces events and de- livers them to the channel, which stores the events until they are forwarded to the sink.
- A Flume installation is made up of a collection of connected agents running in a distributed topology.
- Agents on the edge of the system (co-located on web server machines, for example) collect data and forward it to agents that are responsible for aggregating and then storing the data in its final destination.
- Agents are configured to run a collection of sources and sinks, so using Flume is mainly a configuration exercise in wiring the pieces together
- Download a stable release of the Flume binary distribution from the download page, and unpack the tarball in a suitable location:

  % tar xzf apache-flume-x.y.z-bin.tar.gz

- It's useful to put the Flume binary on your path:

  % export FLUME_HOME=~/sw/apache-flume-x.y.z-bin

Architecture of Flume (Contd.)

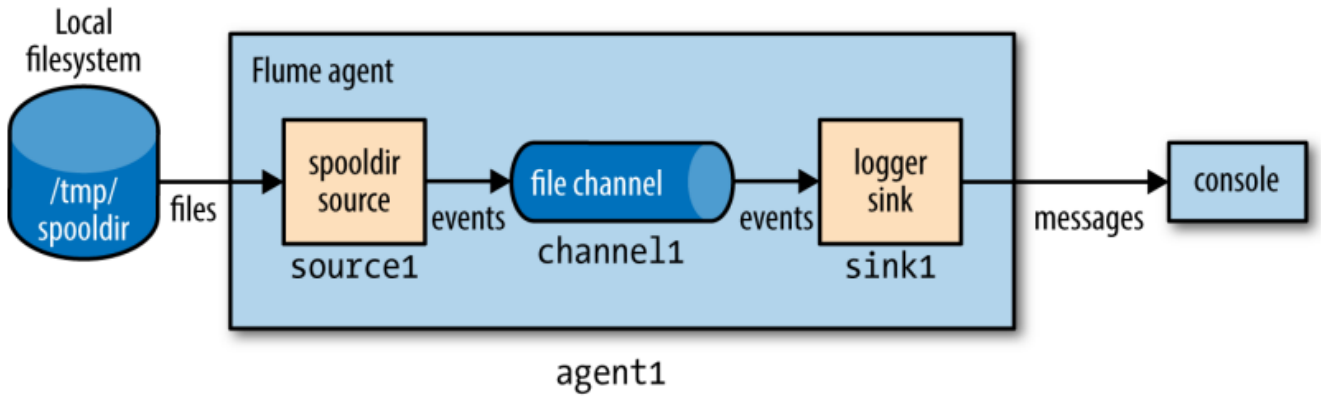
```
agent1.sources = source1
agent1.sinks = sink1
agent1.channels = channel1

agent1.sources.source1.channels = channel1
agent1.sinks.sink1.channel = channel1

agent1.sources.source1.type = spooldir
agent1.sources.source1.spoolDir = /tmp/spooldir

agent1.sinks.sink1.type = logger

agent1.channels.channel1.type = file
```



- Property names form a hierarchy with the agent's name at the top level.
- The names for the different components in an agent are defined at the next level, so for example agent1.sources lists the names of the sources that should be run in agent1 (here it is a single source, source1).
- Similarly, agent1 has a sink (sink1) and a channel (channel1).
- The sink is a logger sink for logging events to the console.
- It too must be connected to the channel (with the agent1.sinks.sink1.channel property).1
- The channel is a file channel, which means that events in the channel are persisted to disk for durability.

Architecture of Flume (Contd.)

- Before running the example, we need to create the spooling directory on the local file- system:

```
% mkdir /tmp/spooldir
```

- Then we can start the Flume agent using the flume-ng command:

```
% flume-ng agent \ --conf-file spool-to-logger.properties \
--name agent1 \
--conf $FLUME_HOME/conf \
-Dflume.root.logger=INFO, console
```

- The Flume properties file is specified with the --conf-file flag.
- The agent's name must also be passed in with --name (since a Flume properties file can define several agents, we have to say which one to run)
- The --conf flag tells Flume where to find its general configuration, such as environment settings.

Architecture of Flume (Contd.)

- In a new terminal, create a file in the spooling directory.
- The spooling directory source expects files to be immutable.
- To prevent partially written files from being read by the source, we write the full contents to a hidden file.
- Then, we do an atomic rename so the source can read it:

```
% echo "Hello Flume" > /tmp/spooldir/.file1.txt
% mv /tmp/spooldir/.file1.txt /tmp/spooldir/file1.txt
```

- Back in the agent's terminal, we see that Flume has detected and processed the file:
- Preparing to move file /tmp/spooldir/file1.txt to /tmp/spooldir/file1.txt.COMPLETED
- Event: { headers:{} body: 48 65 6C 6C 6F 20 46 6C 75 6D 65 Hello Flume }

Transactions and Reliability

Transactions and Reliability

- Flume uses separate transactions to guarantee delivery from the source to the channel and from the channel to the sink.
- In the example in the previous section, the spooling directory source creates an event for each line in the file.
- The source will only mark the file as completed once the transactions encapsulating the delivery of the events to the channel have been successfully committed.
- Similarly, a transaction is used for the delivery of the events from the channel to the sink.
- If for some unlikely reason the events could not be logged, the transaction would be rolled back, and the events would remain in the channel for later redelivery
- The channel we are using is a file channel, which has the property of being durable: once an event has been written to the channel, it will not be lost, even if the agent restarts.
- Flume also provides a memory channel that does not have this property, since events are stored in memory.
  - With this channel, events are lost if the agent restarts. Depending on the application, this might be acceptable.
  - The trade-off is that the memory channel has higher throughput than the file channel.

Transactions and Reliability (Contd.)

- The overall effect is that every event produced by the source will reach the sink.
- The major caveat here is that every event will reach the sink at least once—that is, duplicates are possible.
- Duplicates can be produced in sources or sinks: for example, after an agent restart, the spooling directory source will redeliver events for an uncompleted file, even if some or all of them had been committed to the channel before the restart.
- After a restart, the logger sink will re-log any event that was logged but not committed (which could happen if the agent was shut down between these two operations).

Transactions and Reliability (Contd.)

Batching

- For efficiency, Flume tries to process events in batches for each transaction, where possible, rather than one by one.
- Batching helps file channel performance in particular, since every transaction results in a local disk write and fsync call.
- The batch size used is determined by the component in question and is configurable in many cases.
- For example, the spooling directory source will read files in batches of 100 lines. (This can be changed by setting the batch Size property.)
- Similarly, the Avro sink will try to read 100 events from the channel before sending them over RPC, although it won't block if fewer are available.

Transactions and Reliability (Contd.)

The HDFS Sink

- The point of Flume is to deliver large amounts of data into a Hadoop data store,
- So, let's look at how to configure a Flume agent to deliver events to an HDFS sink.
- The configuration in updates the previous example to use an HDFS sink.
- The only two settings that are required are the sink's type (hdfs) and hdfs.path, which specifies the directory where files will be placed (if, like here, the filesystem is not specified in the path, it's determined in the usual way from Hadoop's fs.defaultFS property).
- We've also specified a meaningful file prefix and suffix, and instructed Flume to write events to the files in text format

Transactions and Reliability (Contd.)

The HDFS Sink

```
agent1.sources = source1
agent1.sinks = sink1
agent1.channels = channel1

agent1.sources.source1.channels = channel1
agent1.sinks.sink1.channel = channel1

agent1.sources.source1.type = spooldir
agent1.sources.source1.spoolDir = /tmp/spooldir

agent1.sinks.sink1.type = hdfs
agent1.sinks.sink1.hdfs.path = /tmp/flume
agent1.sinks.sink1.hdfs.filePrefix = events
agent1.sinks.sink1.hdfs.fileSuffix = .log
agent1.sinks.sink1.hdfs.inUsePrefix = _
agent1.sinks.sink1.hdfs.fileType = DataStream

agent1.channels.channel1.type = file
```

- Restart the agent to use the spool-to-hdfs. properties configuration, and create a new file in the spooling directory:
  % echo -e "Hello\nAgain" > /tmp/spooldir/.file2.txt
  % mv /tmp/spooldir/.file2.txt /tmp/spooldir/file2.txt
- Events will now be delivered to the HDFS sink and written to a file.
- Files in the process of being written to have a .tmp in-use suffix added to their name to indicate that they are not yet complete.
- In this example, we have also set hdfs.inUsePrefix to be _ (underscore; by default it is empty), which causes files in the process of being written to have that prefix added to their names.
- This is useful since MapReduce will ignore files that have a _ prefix.
- So, a typical temporary filename would be events. 1399295780136.log.tmp; the number is a timestamp generated by the HDFS sink.

Transactions and Reliability (Contd.)

The HDFS Sink

- A file is kept open by the HDFS sink until it has either been open for a given time (default 30 seconds, controlled by the hdfs.rollInterval property), has reached a given size (default 1,024 bytes, set by hdfs.rollSize), or has had a given number of events written to it (default 10, set by hdfs.rollCount).
- If any of these criteria are met, the file is closed and its in-use prefix and suffix are removed.
- New events are written to a new file (which will have an in-use prefix and suffix until it is rolled).
- After 30 seconds, we can be sure that the file has been rolled and we can take a look at its contents:

```
% hadoop fs -cat /tmp/flume/events.1399295780136.log
Hello
Again
```

- The HDFS sink writes files as the user who is running the Flume agent, unless the hdfs.proxyUser property is set, in which case files will be written as that user.

Transactions and Reliability (Contd.)

Partitioning and Interceptors

- Large datasets are often organized into partitions, so that processing can be restricted to particular partitions if only a subset of the data is being queried.
- For Flume event data, it's very common to partition by time.
- A process can be run periodically that transforms completed partitions (to remove duplicate events, for example).
- It's easy to change the example to store data in partitions by setting hdfs.path to include subdirectories that use time format escape sequences:

  agent1.sinks.sink1.hdfs.path = /tmp/flume/year=%Y/month=%m/day=%d

- Here we have chosen to have day-sized partitions, but other levels of granularity are possible, as are other directory layout schemes.
- The partition that a Flume event is written to is determined by the timestamp header on the event.
- Events don't have this header by default, but it can be added using a Flume interceptor.
- Interceptors are components that can modify or drop events in the flow; they are attached to
- sources, and are run on events before the events have been placed in a channel.

Transactions and Reliability (Contd.)

Partitioning and Interceptors

- The following extra configuration lines add a timestamp interceptor to source1, which adds a timestamp header to every event produced by the source:

```
agent1.sources.source1.interceptors = interceptor1
agent1.sources.source1.interceptors.interceptor1.type = timestamp
```

- Using the timestamp interceptor ensures that the timestamps closely reflect the times at which the events were created.
- For some applications, using a timestamp for when the event was written to HDFS might be sufficient—although, be aware that when there are multiple tiers of Flume agents there can be a significant difference between creation time and write time, especially in the event of agent downtime
- For these cases, the HDFS sink has a setting, hdfs.useLocal TimeStamp, that will use a timestamp generated by the Flume agent running the HDFS sink.

Transactions and Reliability (Contd.)

File Formats

- It's normally a good idea to use a binary format for storing your data in, since the resulting files are smaller than they would be if you used text.
- For the HDFS sink, the file format used is controlled using hdfs.fileType and a combination of a few other properties.
- If unspecified, hdfs.fileType defaults to SequenceFile, which will write events to a sequence file with Long Writable keys that contain the event timestamp (or the current time if the timestamp header is not present) and Bytes Writable values that contain the event body.
- It's possible to use Text Writable values in the sequence file instead of Bytes Writable by setting hdfs.writeFormat to Text.
- The configuration is a little different for Avro files.
- The hdfs.fileType property is set to DataStream, just like for plain text.
- Additionally, serializer (note the lack of an hdfs. prefix) must be set to avro_event To enable compression, set the serializer.compressionCodec property.

Transactions and Reliability (Contd.)

File Formats

- Here is an example of an HDFS sink configured to write Snappy-compressed Avro files:

```
agent1.sinks.sink1.type = hdfs
agent1.sinks.sink1.hdfs.path = /tmp/flume
agent1.sinks.sink1.hdfs.filePrefix = events
agent1.sinks.sink1.hdfs.fileSuffix = .avro
agent1.sinks.sink1.hdfs.fileType = DataStream
agent1.sinks.sink1.serializer = avro_event
agent1.sinks.sink1.serializer.compressionCodec = snappy
```

- An event is represented as an Avro record with two fields: headers, an Avro map with string values, and body, an Avro bytes field.
- If you want to use a custom Avro schema, there are a couple of options. If you have Avro in-memory objects that you want to send to Flume, then the Log4jAppender is appropriate.
- It allows you to log an Avro Generic, Specific, or Reflect object using a log4j Logger and send it to an Avro source running in a Flume agent.

Transactions and Reliability (Contd.)

File Formats

- In this case, the serializer property for the HDFS sink should be set to org.apache.flume.sink.hdfs.AvroEventSerializer$Builder, and the Avro schema set in the header
- Alternatively, if the events are not originally derived from Avro objects, you can write a custom serializer to convert a Flume event into an Avro object with a custom schema.
- The helper class AbstractAvroEventSerializer in the org.apache.flume.serialization package is a good starting point

Fanout

- Fan out is the term for delivering events from one source to multiple channels, so they reach multiple sinks.
- For example, the configuration in Example delivers events to both an HDFS sink (sink1a via channel1a) and a logger sink (sink1b via channel1b).
- The key change here is that the source is configured to deliver to multiple channels by setting agent1.sources.source1.channels to a space-separated list of channel names, channel1a and channel1b.
- This time, the channel feeding the logger sink (channel1b) is a memory channel, since we are logging events for debugging purposes and don't mind losing events on agent restart.
- Also, each channel is configured to feed one sink, just like in the previous examples.

```
agent1.sources = source1
agent1.sinks = sink1a sink1b
agent1.channels = channel1a channel1b

agent1.sources.source1.channels = channel1a channel1b
agent1.sinks.sink1a.channel = channel1a
agent1.sinks.sink1b.channel = channel1b

agent1.sources.source1.type = spooldir
agent1.sources.source1.spoolDir = /tmp/spooldir

agent1.sinks.sink1a.type = hdfs
agent1.sinks.sink1a.hdfs.path = /tmp/flume
agent1.sinks.sink1a.hdfs.filePrefix = events
agent1.sinks.sink1a.hdfs.fileSuffix = .log
agent1.sinks.sink1a.hdfs.fileType = DataStream

agent1.sinks.sink1b.type = logger

agent1.channels.channel1a.type = file
agent1.channels.channel1b.type = memory
```

Fanout (Contd.)

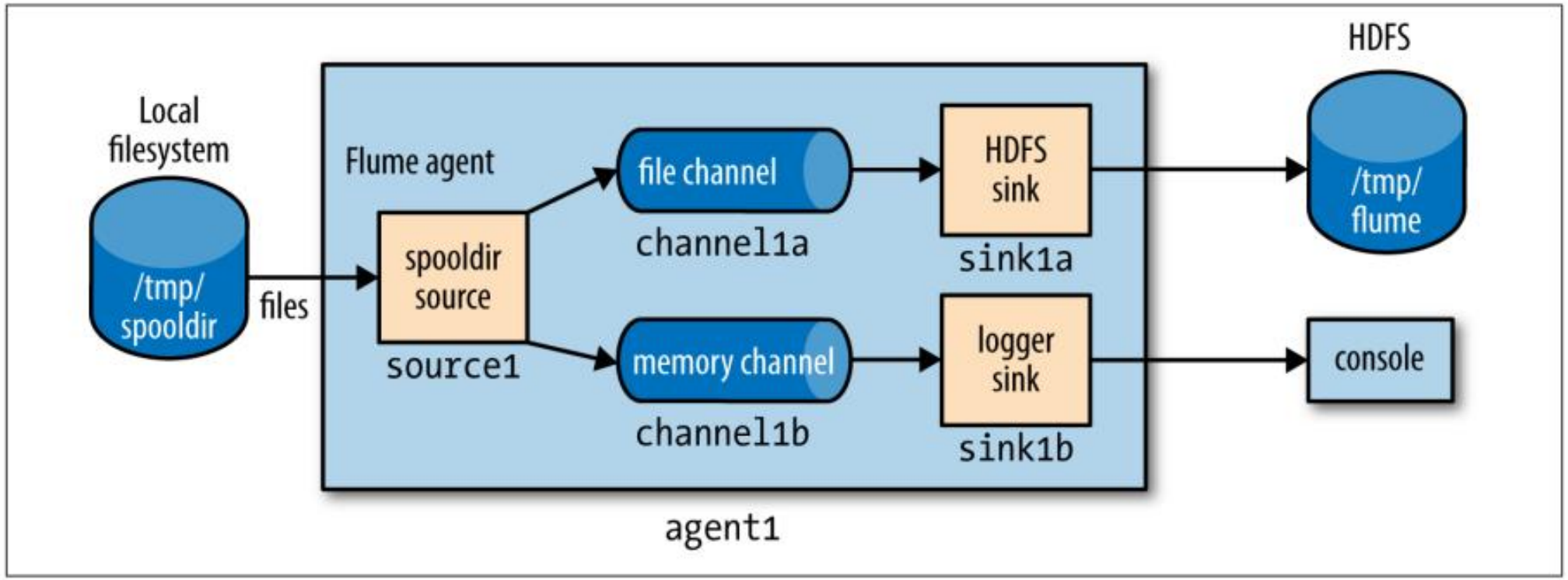


Figure 14-2. Flume agent with a spooling directory source and fanning out to an HDFS sink and a logger sink

Fanout (Contd.)

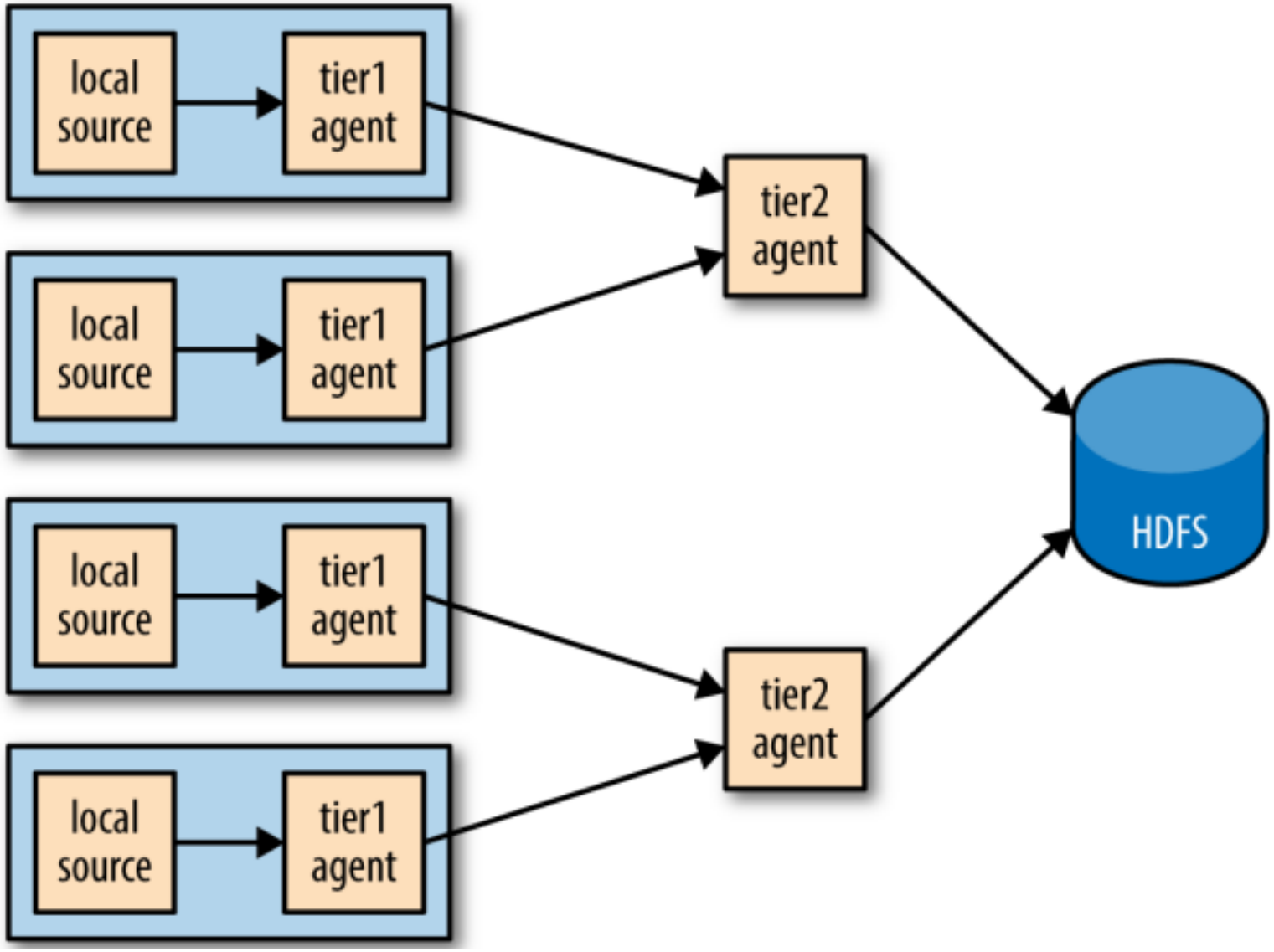
Delivery Guarantees

- Flume uses a separate transaction to deliver each batch of events from the spooling directory source to each channel.
- In this example, one transaction will be used to deliver to the channel feeding the HDFS sink, and then another transaction will be used to deliver the same batch of events to the channel for the logger sink.
- If either of these transactions fails (if a channel is full, for example), then the events will not be removed from the source and will be retried later.
- To do this, we set the selector. Optional property on the source, passing it a space-separated list of channels: agent1.sources.source1.selector.optional = channel

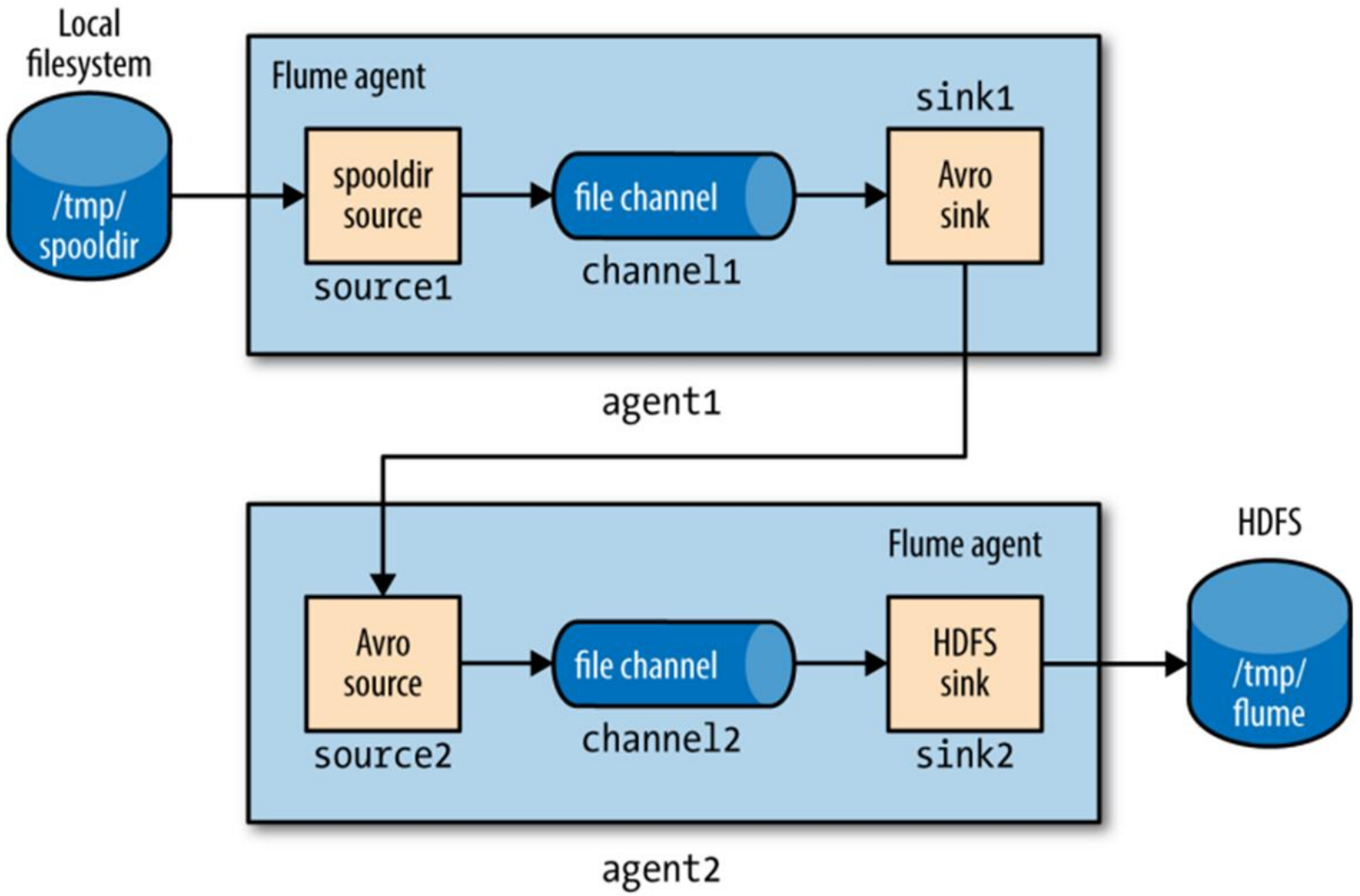
Fanout (Contd.)

- Also, if needed, files can be rolled more often since they are being fed by a larger number of nodes, leading to a reduction between the time when an event is created and when it's available for analysis.
- Aggregating Flume events is achieved by having tiers of Flume agents.
- The first tier collects events from the original sources (such as web servers) and sends them to a smaller set of agents in the second tier, which aggregate events from the first tier before writing them to HDFS
- Further tiers may be warranted for very large numbers of source nodes.
- Tiers are constructed by using a special sink that sends events over the network, and a corresponding source that receives events.
- The Avro sink sends events over Avro RPC to an Avro source running in another Flume agent.
- There is also a Thrift sink that does the same thing using Thrift RPC, and is paired with a Thrift source

Fanout (Contd.)



Fanout (Contd.)



Two Flume agents connected by an Avro sink-source pair

- Each agent is run independently, using the same --conf-file parameter but different agent --name parameters:
- % flume-ng agent --conf-file spool-to-hdfs-tiered.properties --name agent1 ... and:
- % flume-ng agent --conf-file spool-to-hdfs-tiered.properties --name agent2 ...

Fanout (Contd.)

Delivery Guarantee

- Flume uses transactions to ensure that each batch of events is reliably delivered from a source to a channel, and from a channel to a sink.
- In the context of the Avro sink-source connection, transactions ensure that events are reliably delivered from one agent to the next.
- The operation to read a batch of events from the file channel in agent1 by the Avro sink will be wrapped in a transaction.
- The transaction will only be committed once the Avrosink has received the (synchronous) confirmation that the write to the Avro source's RPC endpoint was successful.
- This confirmation will only be sent once agent2's transaction wrapping the operation to write the batch of events to its file channel has been successfully committed.
- Thus, the Avro sink-source pair guarantees that an event is delivered from one Flume agent's channel to another Flume agent's channel (at least once).
- If either agent is not running, then clearly events cannot be delivered to HDFS.
- For example, if agent1 stops running, then files will accumulate in the spooling directory, to be processed once agent1 starts up again.

Fanout (Contd.)

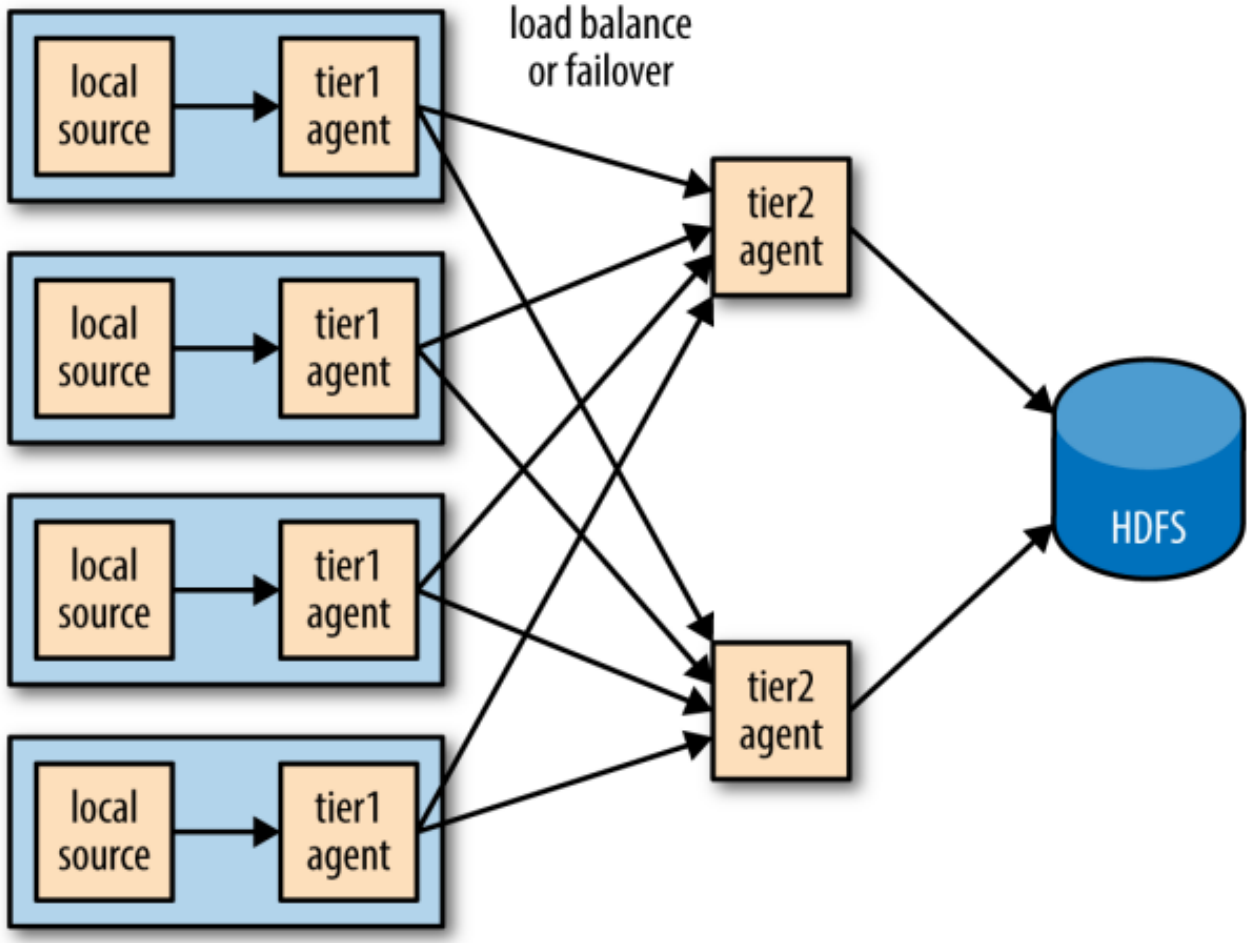
Delivery Guarantee

- Also, any events in an agent's own file channel at the point the agent stopped running will be available on restart, due to the durability guarantee that file channel provides.
- If agent2 stops running, then events will be stored in agent1's file channel until agent2 starts again.
- Note, however, that channels necessarily have a limited capacity; if agent1's channel fills up while agent2 is not running, then any new events will be lost
- By default, a file channel will not recover more than one million events (this can be overridden by its capacity property), and it will stop accepting events if the free disk space for its checkpoint directory falls below 500 MB (controlled by the minimumRequiredSpace property).
- Both these scenarios assume that the agent will eventually recover, but that is not always the case (if the hardware it is running on fails, for example).
- If agent1 doesn't recover, then the loss is limited to the events in its file channel that had not been delivered to agent2 before agent1 shut down.

Fanout (Contd.)

Sink Groups

- A sink group allows multiple sinks to be treated as one, for failover or load-balancing purposes
- If a second-tier agent is unavailable, then events will be delivered to another second-tier agent and on to HDFS without disruption.
- To configure a sink group, the agent's sink groups property is set to define the sink group's name; then the sink group lists the sinks in the group, and also the type of the sink processor, which sets the policy for choosing a sink



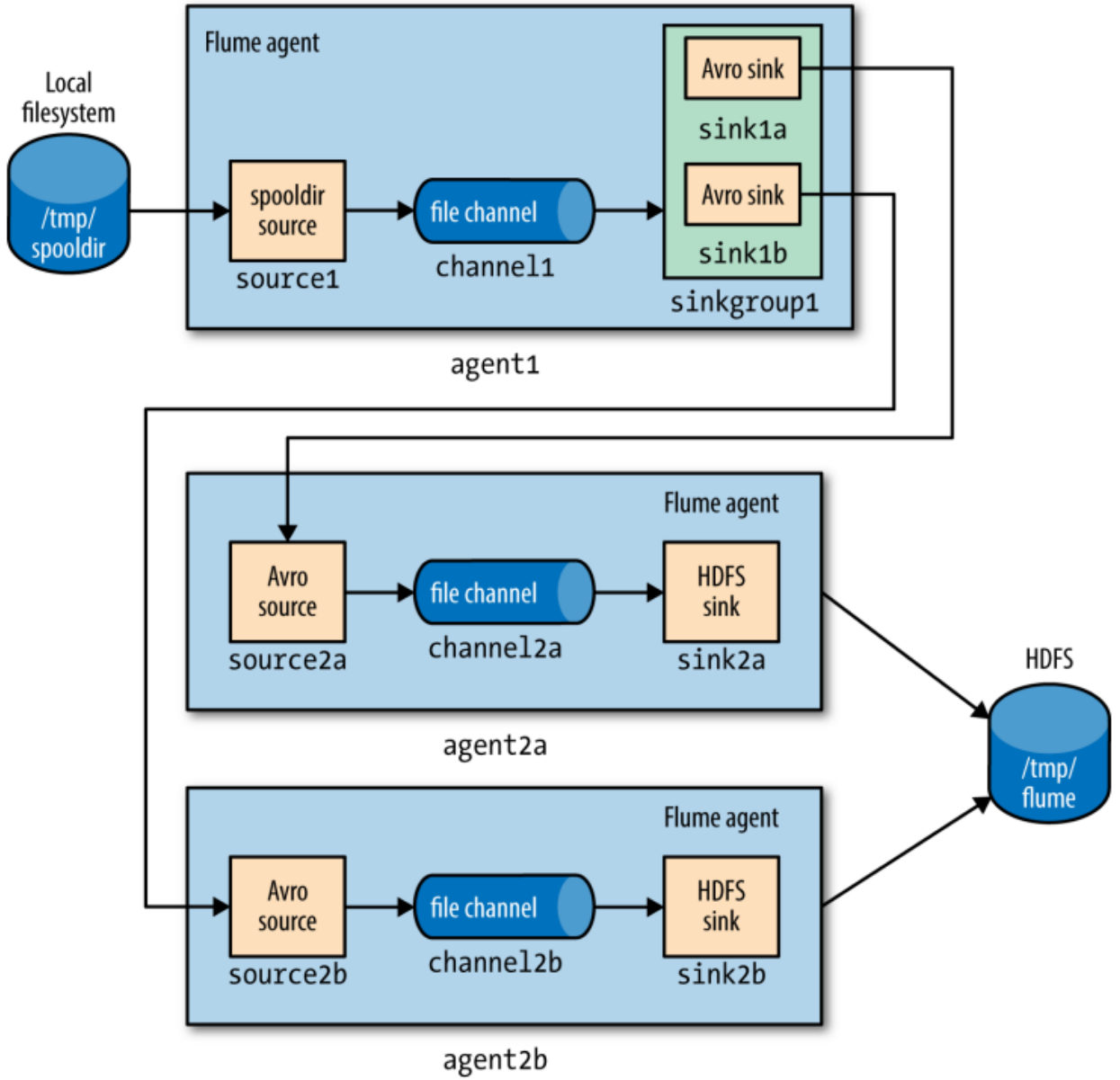
Fanout (Contd.)

Sink Groups

- The processor type is set to load_balance, which attempts to distribute the event flow over both sinks in the group, using a round-robin selection mechanism (you can change this using the processor.selector property).
- If a sink is unavailable, then the next sink is tried; if they are all unavailable, the event is not removed from the channel, just like in the single sink case.
- By default, sink unavailability is not remembered by the sink processor, so failing sinks are retried for every batch of events being delivered.
- This can be inefficient, so we have set the processor.backoff property to change the behavior so that failing sinks are blacklisted for an exponentially increasing timeout period (up to a maximum period of 30 seconds, controlled by processor.selector.maxTimeOut).

Fanout (Contd.)

Sink Groups



Fanout (Contd.)

Integrating Flume with Applications

- An Avro source is an RPC endpoint that accepts Flume events, making it possible to write an RPC client to send events to the endpoint, which can be embedded in any application that wants to introduce events into Flume.
- The Flume SDK is a module that provides a Java RpcClient class for sending Event objects to an Avro endpoint (an Avro source running in a Flume agent, usually in an- other tier).
- Clients can be configured to fail over or load balance between endpoints, and Thrift endpoints (Thrift sources) are supported too.
- The Flume embedded agent offers similar functionality: it is a cut-down Flume agent that runs in a Java application. It has a single special source that your application sends Flume Event objects to by calling a method on the EmbeddedAgent object; the only sinks that are supported are Avro sinks, but it can be configured with multiple sinks for failover or load balancing.


thank
you